⇄ Gust First-Person Controller

Unity 6.3 LTS (6000.3.261f1)
By Woyboy

Hello, and welcome to Gust FPC, also known as Gust First-Person Controller! This is a simple first person controller utilizing Unity's CharacterController Component in Unity 6! This package includes a **simple first-person character controller and an interactable system** utilizing Unity's **new Input System**. Below are some of your questions answered, and some of the helpful methods that you might find yourself using in the development of your game.

FAQ

What is this controller intended for?

- Anything! The controller can be used for anything you please. There is no limit to the controller, because it is extremely customizable and you can change the script to whatever you want. This package is meant to be a fast way of getting your first person controller down on the spot! My game development mainly consists of horror games, so it may have some intention with horror game development

Why use a CharacterController component but not a Rigidbody?

- Unity 6 has changed this component to make character controllers much more easier to implement. As compared to rigidbodies, I believe that this has a much more realistic appeal to it than a generic rigidbody controller.

Why should I use your terrible controller compared to others?

- I actually have no answer for that. There are a lot of existing controllers as of today that you can browse and use. But this is mine and mine alone!

Why did you call it Gust FPC?

- Yamanin Zephyr is my oshi. Embrace the gusts of winds Zephyr!

How to Set Up

Download the GitHub package manually. You can choose to download the entire project or download the UnityPackage file. I recommend the Unity Package as it sets up everything for you easily and *should* include all the necessary project settings. Once acquired, go to the /Scenes folder and find your BlankFPCScene or DemoScene to copy and paste your GustFPC. Then assign the necessary layers/values needed for the controller to work properly. **Click on Player, Check your Player Input to see if you have the right input (Although one is provided for you), Check the PlayerInteraction.cs and assign the Interaction Layer (Everything is set by default), lastly check the Sound Settings and create some layers. Default ground Layers are Grass, Metal, Snow**

TL:DR

- Download UnityPackage file or the entire thing
- Assign Player Input in the Player.cs script, (One is provided for you so this can be skipped)
- Assign a Interaction Layer in Player.cs (Everything is enabled by default, safe to skip)
- Create 3 layers based off the Ground Sounds, which you can find in Sound Settings of Player.cs (**Grass, Metal, Snow**)

How to add onto existing scenes?

- Go to /**Scenes** and find the **DemoScene** or the **BlankFPCScene**
- **Copy** the GustFPC Player GameObject
- **Paste** the prefab into your existing scene.
- Or just go to /Prefabs and drag and drop the existing player prefab
- And you're done!

Controller Architecture

If you want to modify the script or add onto the controller, here are a few things to keep in mind. The controller works independently with the Player.cs script being the main parent. Followed under that is the rest of the independent controllers such as PlayerMovement.cs These scripts all require one class to function which is the PlayerDependency.cs class which holds all the necessary values to make your character move. Means that PlayerDependency.cs should never be removed and recommended to be placed along with the other scripts in one gameObject.

Additionally, the character controller is designed in mind with the idea of expanding into multiplayer development. I already have an existing **MultiplayerCore** package available in my GitHub if you're interested in that. But don't expect too much of it.

The Interaction System:

The interaction system is straightforward. The PlayerInteractionController.cs casts a raycast searching for the interactables based on the layers you select. Once found, it searches for the interface **IInteractable**, or if it has an Interactable inherit.

Useful Methods:

There are a good amount of built-in methods that can help with your development, but just know that you can always add more. Everything is already set up for you perfectly! (Hopefully)
The following methods are found in Player.cs and don't need an explanation. Then the interaction is checked through the InputHandler.cs and executes the void Interact();

TeleportPlayer(Transform targetLocation) // Teleports the player to a location

LockPlayer(bool showCursor = false) // Lock player controls completely

UnlockPlayer() // Undo the LockPlayer()

SetCursorVisible(bool visible) // Locking and hiding/showing cursor, careful when using this

Thank you Zephyr (Umamusume Pretty Derby)

